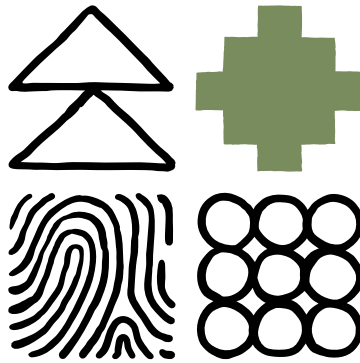


构建高效的 Agent

Building effective agents

Anthropic · 2024-12-19 · 原文 / Source

完整译文与英文逐段对照。依据你提供的网页存档翻译，保留原文的观点、例子与限定条件。图表（含图注）、代码及其中的文字维持原文；网站导航、订阅表单和站点页脚不属于文章正文。中文为学习用非官方译文。



我们曾与来自各行各业的数十个团队合作，构建基于 LLM 的 Agent。我们始终发现，最成功的实现采用的都是简单、可组合的模式，而不是复杂的框架。

注：自 2024 年 12 月以来，本文所描述的工具生态已有许多变化。关于我们目前采用的方法，请参阅[我们如何构建 Claude Managed Agents](#)，以及[Managed Agents 文档](#)。

过去一年，我们曾与来自各行各业的数十个团队合作，构建基于大语言模型（LLM）的 Agent。我们始终发现，最成功的实现并没有使用复杂框架或专用库，而是采用了简单、可组合的模式。

在本文中，我们将分享与客户合作以及自行构建 Agent 的经验，并就如何构建有效的 Agent 向开发者提供实用建议。

什么是 Agent？

“Agent”可以有多种定义。有些客户将 Agent 定义为完全自主的系统：它们能够长时间独立运行，使用各种工具完成复杂任务。另一些客户则用这个词描述约束更明确、遵循预定义工作流的实现。在 Anthropic，我们把这些不同形式都归为**智能体系统（agentic systems）**，但在架构上对**工作流**和**Agent**做了一个重要区分：

- **工作流**是通过预定义的代码路径来编排 LLM 和工具的系统。
- **Agent** 则是由 LLM 动态决定自身流程和工具使用方式的系统，LLM 掌握着如何完成任务的控制权。

下面，我们将详细探讨这两类智能体系统。在附录 1（“Agent 实践”）中，我们会介绍两个领域，客户在这些领域使用此类系统时发现了尤其突出的价值。

何时应该使用 Agent，何时不应该使用

使用 LLM 构建应用时，我们建议寻找尽可能简单的解决方案，仅在需要时增加复杂性。这可能意味着完全不必构建智能体系统。智能体系统往往以更高的延迟和成本来换取更好的任务表现，你应当考虑在什么情况下，这种权衡才是合理的。

如果确实需要更高的复杂性，对于定义明确的任务，工作流能提供可预测性和一致性；而当你需要大规模地实现灵活处理和模型驱动的决策时，Agent 是更好的选择。不过，对许多应用来说，使用检索和上下文内示例来优化单次 LLM 调用，通常就已经足够。

何时以及如何使用框架

许多框架能够让智能体系统更容易实现，包括：

- Claude Agent SDK；
- AWS 的 Strands Agents SDK；
- Rivet，一种采用拖放式图形用户界面的 LLM 工作流构建工具；以及
- Vellum，另一种用于构建和测试复杂工作流的图形界面工具。

这些框架通过简化调用 LLM、定义和解析工具、串联多次调用等常规底层任务，让你更容易上手。但它们往往也会增加额外的抽象层，可能遮蔽底层的提示词和响应，使调试更加困难。在更简单的配置已经足够时，它们也容易让人忍不住增加复杂性。

我们建议开发者先从直接使用 LLM API 开始：许多模式只需几行代码就能实现。如果确实使用框架，请确保自己理解底层代码。对内部机制做出错误假设，是客户出错的常见原因。

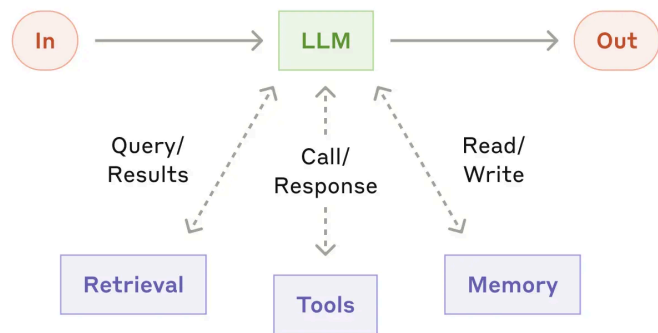
部分实现示例可参见我们的实践示例集。

基础构件、工作流与 Agent

本节将探讨我们在生产环境中见到的智能体系统常用模式。我们从基础构件——增强型 LLM——开始，逐步增加复杂性，从简单的组合式工作流，一直讲到自主 Agent。

基础构件：增强型 LLM

智能系统的基本构件，是通过检索、工具、记忆等能力加以增强的 LLM。我们目前的模型能够主动使用这些能力：自行生成搜索查询、选择合适的工具，并决定保留哪些信息。



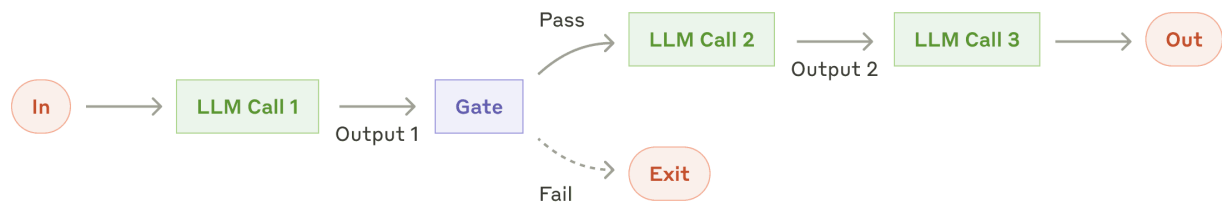
The augmented LLM

我们建议在实现时重点关注两个方面：根据具体使用场景定制这些能力，并确保它们为 LLM 提供简单易用、文档完善的接口。实现这些增强能力的方法有很多，其中一种是我们最近发布的模型上下文协议（Model Context Protocol）。它让开发者只需一个简单的客户端实现，就能接入不断发展的第三方工具生态。

在本文余下部分，我们假设每次 LLM 调用都能访问这些增强能力。

工作流：提示链

提示链把一个任务分解为一系列步骤，每次 LLM 调用都处理上一次调用的输出。你可以在任意中间步骤加入程序化检查（参见下图中的“gate”），确保流程仍沿着正确方向推进。



The prompt chaining workflow

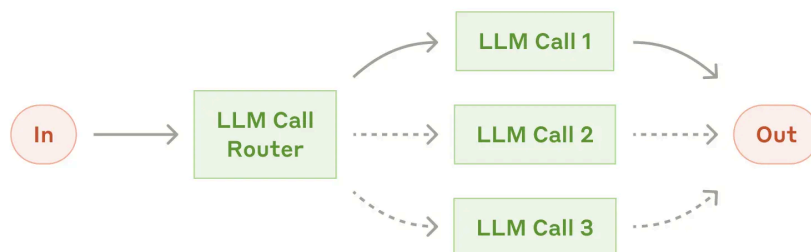
何时使用这种工作流：当任务可以轻松、清晰地拆分为固定子任务时，这种工作流非常适合。其主要目标是让每次 LLM 调用承担更简单的任务，以增加延迟为代价换取更高的准确性。

适合使用提示链的示例：

- 先生成营销文案，再将其翻译成另一种语言。
- 先编写文档提纲，检查提纲是否满足特定标准，再根据提纲撰写文档。

工作流：路由

路由对输入进行分类，并将其交给专门的后续任务处理。这种工作流能够分离不同职责，并构建更有针对性的提示词。如果没有这种工作流，针对一类输入做出的优化，可能会损害处理其他输入时的表现。



The routing workflow

何时使用这种工作流：对于包含明确不同类别、且分开处理效果更好的复杂任务，如果 LLM 或更传统的分类模型 / 算法能够准确完成分类，路由就很适合。

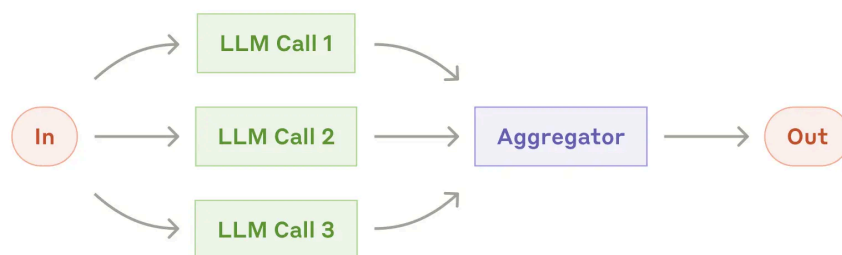
适合使用路由的示例：

- 将不同类型的客户服务咨询（一般问题、退款请求、技术支持）分流到不同的下游流程、提示词和工具。
- 将简单 / 常见的问题路由到 Claude Haiku 4.5 这类较小、成本效益较高的模型，将困难 / 不常见的问题路由到 Claude Sonnet 4.5 这类能力更强的模型，以获得最佳表现。

工作流：并行化

有时可以让多个 LLM 同时处理一个任务，再通过程序汇总它们的输出。这种称为“并行化”的工作流主要有两种形式：

- **分工：**将任务拆分为独立的子任务，并行执行。
- **投票：**多次执行同一个任务，得到多样化的输出。



The parallelization workflow

何时使用这种工作流：当拆分后的子任务可以并行执行以提高速度，或者需要通过多个视角或多次尝试来得到更可信的结果时，并行化很有效。对于需要考虑多个方面的复杂任务，让每个方面分别由一次 LLM 调用处理，通常能取得更好的表现，因为这样可以把注意力集中在各个具体方面。

适合使用并行化的示例：

- **分工：**

- 实施安全护栏：一个模型实例处理用户查询，另一个则筛查其中是否存在不当内容或请求。这通常比让同一次 LLM 调用同时处理安全护栏和主要响应效果更好。
- 将 LLM 表现的评测自动化：每次 LLM 调用，分别评测模型在给定提示词上表现的一个不同方面。

- **投票：**

- 审查一段代码是否存在漏洞：使用多个不同的提示词进行审查，发现问题时就对代码做出标记。
- 评估给定内容是否不当：使用多个提示词评价不同方面，或者设置不同的投票阈值，以平衡误报和漏报。

工作流：编排者与执行者

在“编排者与执行者”工作流中，一个中央 LLM 动态拆解任务，将任务委派给作为执行者的 LLM，并综合它们的结果。



The orchestrator-workers workflow

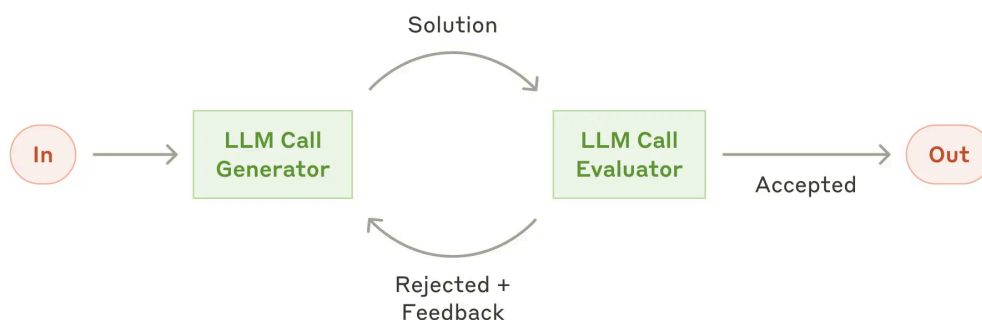
何时使用这种工作流：这种工作流很适合那些无法预先确定所需子任务的复杂任务。例如，在编程任务中，需要修改多少个文件，以及每个文件需要做什么修改，通常都取决于具体任务。虽然它在结构上与并行化类似，但关键区别在于灵活性：子任务不是预先定义的，而是由编排者根据具体输入决定的。

适合使用“编排者与执行者”的示例：

- 每次需要对多个文件进行复杂修改的编程产品。
- 需要从多个来源收集和分析信息，以寻找可能相关内容的搜索任务。

工作流：评估者与优化者

在“评估者与优化者”工作流中，一次 LLM 调用生成响应，另一次调用提供评测和反馈，两者循环进行。



The evaluator-optimizer workflow

何时使用这种工作流：当我们拥有明确的评测标准，而且迭代改进能够带来可衡量的价值时，这种工作流尤其有效。判断是否适用有两个信号：第一，人类清楚表达反馈后，LLM 的响应确实能够改善；第二，LLM 也能提供这样的反馈。这类似于人类作者为打磨一篇文档而经历的反复写作过程。

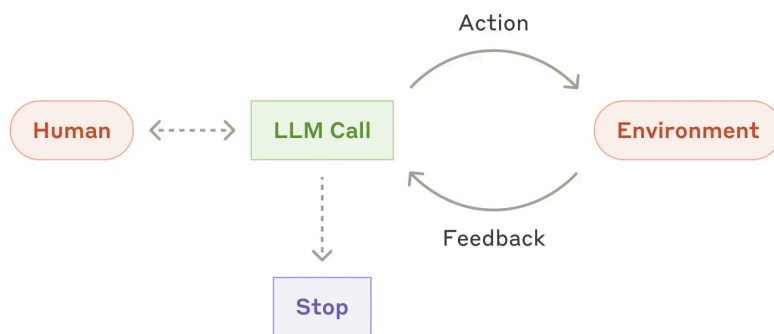
适合使用“评估者与优化者”的示例：

- 文学翻译：负责翻译的 LLM 起初可能没有捕捉到某些细微含义，而负责评估的 LLM 可以提出有益的批评意见。
- 复杂搜索任务：需要经过多轮搜索与分析，才能收集到全面的信息，由评估者决定是否有必要进一步搜索。

Agent

随着 LLM 在理解复杂输入、推理与规划、可靠使用工具、从错误中恢复等关键能力上日益成熟，Agent 正开始出现在生产环境中。Agent 通过接收人类用户的指令，或与用户互动讨论，开始工作。一旦任务明确，Agent 就会独立规划和行动，并可能再次向人类寻求更多信息或判断。在执行过程中，Agent 必须在每一步都从环境中获得“真实依据”（例如工具调用结果或代码执行结果），以评估自身进展，这一点至关重要。随后，Agent 可以在检查点或遇到阻碍时暂停，等待人类反馈。任务通常在完成时终止，但也经常会设置停止条件（例如最大迭代次数），以保持控制。

Agent 能够处理复杂任务，但其实现往往很直接。通常，它只是让 LLM 在循环中根据环境反馈使用工具。因此，清晰、周密地设计工具集及其文档至关重要。我们会在附录 2（“为工具做提示工程”）中进一步讨论工具开发的最佳实践。



Autonomous agent

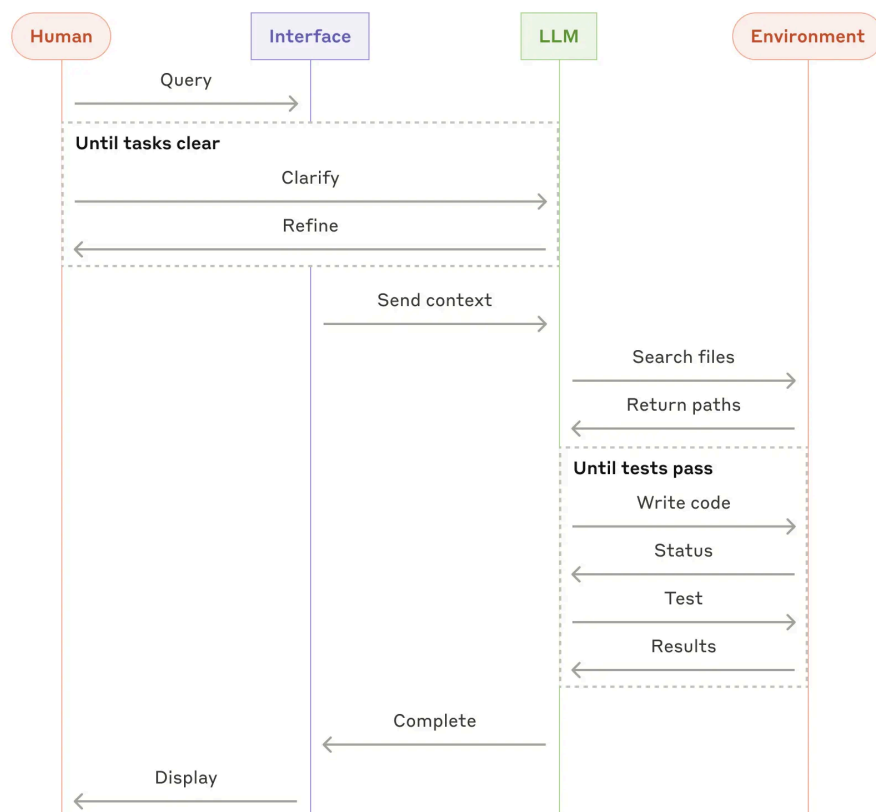
何时使用 Agent：Agent 适用于开放式问题：所需步骤数难以甚至无法预测，也无法将固定路径硬编码下来。LLM 可能运行很多轮，因此，你必须对其决策具有一定程度的信任。Agent 的自主性使其非常适合在可信环境中大规模处理任务。

Agent 的自主性意味着更高的成本，也意味着错误可能不断累积。我们建议在沙箱环境中进行充分测试，并配合适当的安全护栏。

适合使用 Agent 的示例：

以下示例来自我们自己的实现：

- 用于解决 SWE-bench 任务的编程 Agent，这些任务需要根据任务描述修改多个文件；
- 我们的“计算机使用（computer use）”参考实现，其中 Claude 使用计算机完成任务。



High-level flow of a coding agent

组合与定制这些模式

这些构件并不是强制性的规范，而是开发者可以调整、组合，以适应不同使用场景的常见模式。与任何 LLM 功能一样，成功的关键在于衡量表现，并迭代改进实现。再次强调：只有在能够证明复杂性确实改善结果时，你才应考虑增加复杂性。

总结

在 LLM 领域取得成功，并不是要构建最精巧复杂的系统，而是要构建适合自身需求的系统。从简单的提示词开始，通过全面评测来优化它们，只有当更简单的方案无法满足需求时，才加入多步骤的智能体系统。

在实现 Agent 时，我们努力遵循三条核心原则：

1. 保持 Agent 设计的**简单性**。
2. 明确展示 Agent 的规划步骤，优先保证**透明性**。
3. 通过完善的工具文档与测试，精心打造 Agent—计算机接口（ACI）。

框架可以帮助你快速入门，但在走向生产环境的过程中，不必犹豫：可以减少抽象层，使用基础组件来构建系统。遵循这些原则，你就能创建不仅能力强，而且可靠、可维护、受到用户信任的 Agent。

致谢

作者：Erik S. 和 Barry Zhang。本文基于我们在 Anthropic 构建 Agent 的经验，以及客户分享的宝贵见解。对此，我们深表感谢。

附录 1：Agent 实践

在与客户合作的过程中，我们发现了两类尤其有前景的 AI Agent 应用，它们展示了上述模式的实际价值。这两类应用都说明：对于既需要对话又需要行动、具有明确成功标准、能够形成反馈循环，并融入有效人类监督的任务，Agent 能够带来最大的价值。

A. 客户支持

客户支持将人们熟悉的聊天机器人界面，与通过工具集成获得的增强能力结合起来。它天然适合开放程度更高的 Agent，原因如下：

- 支持服务中的交互天然沿着对话流程展开，同时又需要访问外部信息并执行操作；
- 可以集成工具，获取客户数据、订单历史和知识库文章；
- 发放退款、更新工单等操作可以通过程序处理；以及
- 可以根据用户定义的问题解决标准，明确衡量是否成功。

多家公司已经通过一种按使用量计费、且仅在成功解决问题时收费的定价模式，证明了这种方法的可行性，也展示了它们对自家 Agent 有效性的信心。

B. 编程 Agent

软件开发领域已经展现出 LLM 功能的巨大潜力，其能力正从代码补全发展到自主解决问题。Agent 在这个领域尤其有效，原因如下：

- 代码解决方案可以通过自动化测试进行验证；
- Agent 可以使用测试结果作为反馈，迭代改进方案；
- 问题空间定义明确、结构清晰；以及
- 输出质量可以被客观衡量。

在我们自己的实现中，Agent 现在仅凭拉取请求的描述，就能解决 SWE-bench Verified 基准中的真实 GitHub 问题。不过，自动化测试虽然有助于验证功能，但要确保方案符合更广泛的系统要求，人工审查仍然至关重要。

附录 2：为工具做提示工程

无论你在构建哪种智能体系统，工具很可能都是 Agent 的重要组成部分。在我们的 API 中明确指定工具的具体结构和定义，就能让 Claude 与外部服务及 API 交互。当 Claude 响应时，如果它计划调用工具，就会在 API 响应中加入一个工具使用块。你对工具定义和规格投入的提示工程精力，应当与对整体提示词投入的精力一样多。在这篇简短的附录中，我们将介绍如何为工具做提示工程。

同一个操作通常有多种描述方式。例如，要指定对文件的修改，可以编写 diff，也可以重写整个文件。对于结构化输出，可以将代码放在 Markdown 中返回，也可以放在 JSON 中返回。在软件工程中，这类差异只是形式上的差异，彼此可以无损转换。但对于 LLM 来说，有些格式远比其他格式难写。编写 diff 时，在新代码还没写出来之前，就需要在变更块头部确定将有多少行发生变化。与 Markdown 相比，将代码写在 JSON 中还需要额外转义换行符和引号。

关于如何选择工具格式，我们有以下建议：

- 给模型足够的 token 来“思考”，避免它还没想清楚，就把自己写进难以继续的局面。
- 让格式尽量接近模型在互联网文本中自然见过的形式。
- 确保没有格式上的额外“负担”，例如必须准确计算数千行代码的行数，或对写出的所有代码做字符串转义。

一条实用的经验法则是：想一想人们在人机界面（HCI）上投入了多少精力，然后计划投入同样多的精力，来打造良好的 Agent—计算机接口（ACI）。以下是一些具体建议：

- 站在模型的角度思考：根据描述和参数，这个工具的用法是否一目了然，还是需要仔细琢磨？如果你需要琢磨，模型很可能也一样。好的工具定义通常包括用法示例、边界情况、输入格式要求，以及它与其他工具之间清晰的职责边界。
- 你可以怎样修改参数名或描述，让含义更加明确？把这当作是在为团队中的初级开发者编写一份优秀的文档字符串。当存在许多相似工具时，这一点尤其重要。
- 测试模型如何使用你的工具：在我们的工作台中运行大量输入示例，观察模型会犯哪些错误，再迭代改进。
- 为工具做防错设计（Poka-yoke）。调整参数，使错误更难发生。

在为 SWE-bench 构建 Agent 时，我们实际花在优化工具上的时间，比优化整体提示词还要多。例如，我们发现，当 Agent 离开根目录之后，模型在使用采用相对文件路径的工具时会犯错。为解决这个问题，我们将工具改为始终要求绝对文件路径——随后发现，模型采用这种方式时毫无差错。